

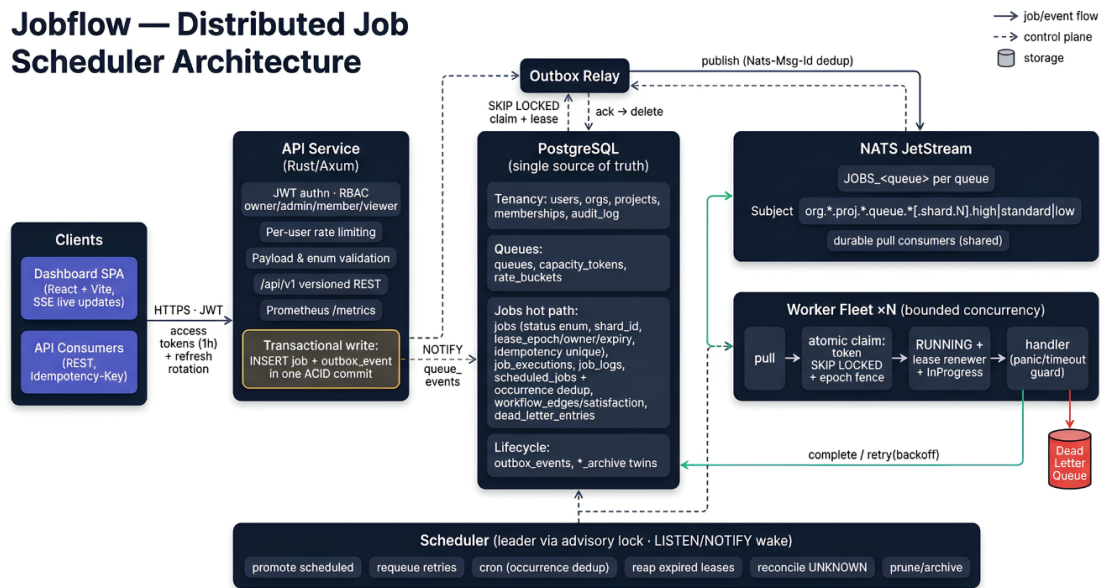
Jobflow

Distributed Job Scheduler

A production-inspired platform for reliable, multi-tenant execution of asynchronous background jobs.

Built with **Rust**, **PostgreSQL**, **NATS JetStream**, and **React**

Jobflow — Distributed Job Scheduler Architecture



Live Demo: <https://jobflow-nlwj.onrender.com>

Source: <https://github.com/Arunkumar06-cmd/distributed-job-scheduler>

Arunkumar K
SDE Intern Assignment

Contents

1	System Overview	2
2	Architecture	2
3	Database Design	3
3.1	Key Design Decisions	3
3.2	Indexes	3
4	Backend Engineering	4
4.1	API Surface	4
4.2	Authentication	4
4.3	Authorization	4
4.4	Validation	4
4.5	Error Handling	4
4.6	Rate Limiting	4
4.7	Audit Trail	4
5	Reliability & Concurrency	4
5.1	Delivery Guarantee	4
5.2	Concurrency Proof Tests	4
5.3	Handler Safety	5
6	Frontend & UX	5
7	API Design	5
7.1	Conventions	5
8	Documentation	6
9	Testing	6
9.1	Key Design Decisions	6
10	Deployment	7
11	Conclusion	7

1 System Overview

Jobflow is a distributed job scheduling platform that reliably executes asynchronous background jobs across multiple workers. It provides durable handoff via a transactional outbox, bounded concurrency via capacity tokens, lease-based fencing against stale workers, automatic retries with configurable backoff strategies, and a dead-letter queue for permanent failures.

The system is organised as a Rust workspace with seven crates:

Crate	Binary	Responsibility
<code>common</code>	library	Shared configuration, error types, ID generation, task-local request-id propagation
<code>domain</code>	library	Pure business logic: job lifecycle state machine, retry policies, cron parsing, queue validation
<code>db</code>	library	All PostgreSQL queries, migrations, per-test database isolation, connection pooling
<code>api</code>	<code>api</code>	REST API server (Axum), authentication, RBAC, rate limiting, SSE/WebSocket, static file serving
<code>outbox</code>	<code>outbox</code>	Transactional outbox relay: claims unpublished events, publishes to JetStream with dedup
<code>scheduler</code>	<code>scheduler</code>	Cron runner, scheduled-job promotion, retry requeue, lease reaping, UNKNOWN reconciliation, archival
<code>worker</code>	<code>worker</code>	JetStream pull consumers, atomic job claiming, handler execution with panic/timeout guards, graceful shutdown

2 Architecture

The system follows a two-authority model: PostgreSQL is the single source of truth for job state, and NATS JetStream handles message delivery to workers. A transactional outbox guarantees that no job is lost between the database commit and the broker publish.

1. A client submits a job via `POST /jobs`. The API validates the request, resolves shard routing, and inserts both the job row and an outbox event in a single ACID transaction.
2. The outbox relay claims unpublished events (`SKIP LOCKED`), publishes them to JetStream with a stable `Nats-Msg-Id` for broker-side deduplication, and deletes the outbox rows on successful publish. Failed publishes receive exponential backoff via a `publish_attempts` column.
3. Workers attach shared durable pull consumers (named by hash of stream+subject, stable across restarts) and receive messages. Each message triggers an atomic claim: the worker locks the queue row, reserves a capacity token (`SKIP LOCKED`), transitions the job from `QUEUED` to `CLAIMED` with an incremented `lease_epoch`, and creates an execution ledger row all in one transaction.
4. The worker transitions to `RUNNING` and spawns a lease renewer that extends the PostgreSQL lease every heartbeat interval. Simultaneously, it sends NATS `InProgress` acknowledgements to extend the JetStream ack-wait.
5. The handler executes under a panic guard and hard timeout. On success, `complete_job` transitions `RUNNING` to `COMPLETED` (fenced by epoch), releases the capacity token, resolves workflow DAG edges, and ACKs the NATS message. On failure, `fail_job` either schedules a retry (`RETRY_WAIT` with computed backoff) or dead-letters the job.

6. A scheduler process (elected via PostgreSQL advisory lock, woken by LISTEN/NOTIFY) promotes scheduled jobs, requeues expired retries, fires cron occurrences with deterministic deduplication, reaps stale leases, reconciles UNKNOWN_EXTERNAL_RESULT jobs, and manages archival housekeeping.

3 Database Design

The schema uses 24 tables across three tiers: hot (active jobs), warm (executions, logs), and cold (archive twins). All enums are PostgreSQL native types mapped 1:1 to Rust enums via `sqlx` derive.

3.1 Key Design Decisions

Lease Epoch Fencing

Each claim increments `lease_epoch`. All subsequent updates (complete, fail, renew) require both the worker ID and the epoch to match. A stale worker's update affects zero rows and is silently fenced.

Capacity Tokens

Each queue has `max_concurrency` token rows. Claiming atomically reserves a free token (SKIP LOCKED); completing or failing releases it. A database trigger keeps tokens in sync when concurrency is resized.

Idempotency

A unique constraint on (`queue_id`, `idempotency_key`) prevents duplicate job creation. The API accepts the key via header (preferred) or body; empty keys are treated as absent.

Cron Deduplication

The `scheduled_occurrences` table uses a composite primary key (`scheduled_job_id`, `fire_time`). Multiple scheduler replicas race to insert; the winner creates the job, losers detect the conflict and skip.

Hot/Cold Separation

Terminal jobs older than `ARCHIVE_AFTER_DAYS` move with their executions, logs, and replayed DLQ entries into `*_archive` twin tables. Un-replayed DLQ entries block archival. Measured throughput: $\sim 4,500$ rows/s.

Migration Idempotency

Every migration is guarded (IF NOT EXISTS, DO \$\$ blocks) and the test harness applies the full set twice on every run, continuously proving that drifted schemas converge.

3.2 Indexes

```
idx_jobs_queued ON jobs(queue_id, priority DESC, created_at) WHERE status = 'QUEUED'
idx_jobs_retry ON jobs(next_retry_at) WHERE status = 'RETRY_WAIT'
idx_jobs_queue_shard_status ON jobs(queue_id, shard_id, status)
idx_jobs_queue_created_at ON jobs(queue_id, created_at DESC)
idx_outbox_claim ON outbox_events(priority DESC, created_at) WHERE published_at IS NULL
idx_executions_job_started ON job_executions(job_id, started_at DESC)
idx_audit_actor_time ON audit_log(actor_id, created_at DESC)
```

Partial indexes match exact query patterns: queue-claim scans, retry-requeue scans, scheduler promotion scans, and outbox relay anti-joins. Composite indexes support rate-limit counting and workflow resolution without full-table scans.

4 Backend Engineering

4.1 API Surface

The stable contract is mounted at `/api/v1/*` with frozen unversioned aliases. 37 endpoints cover authentication, tenancy management, queue configuration, job lifecycle, scheduling, worker monitoring, DLQ management, batch operations, workflow DAGs, and observability.

4.2 Authentication

- Argon2id password hashing with per-hash random salts
- 1-hour access tokens (HS256 JWT) rotated by 30-day refresh tokens
- Typed claims (`typ: access|refresh`) prevent cross-use
- `POST /auth/refresh` issues a fresh pair; stolen access tokens have a maximum blast radius of one hour

4.3 Authorization

Four roles (owner, admin, member, viewer) enforced per-route via a single-round-trip query (`authorize_queue`) that returns the queue row, org ID, and caller's role simultaneously. Owner memberships are immutable through the API to prevent lockout.

4.4 Validation

Layered validation: the `validator` crate for declarative constraints, domain-level validators for cross-field rules (retry bounds, delay ordering), payload object/size checks (256 KiB cap), control-character rejection, and strict enum parsing that rejects unknown values rather than coercing.

4.5 Error Handling

All errors flow through a single `AppError` enum mapped to HTTP status codes. Internal failures (SQL, IO, driver) log full detail at `error!` level but return only `{error:{code:"Internal",message:"internal server error"},request_id}` to clients no stack traces, SQL text, or file paths. A task-local request ID correlates error bodies with the `x-request-id` response header. A middleware rewrites framework-generated plain-text rejections (body-limit, query deserialization) into the same envelope.

4.6 Rate Limiting

Per-user fixed-window limiter (default 600 req/min, configurable) checked after authentication. Per-queue admission limits use a database-backed count of jobs created within a rolling window.

4.7 Audit Trail

Privileged mutations (queue configuration, pause/resume, DLQ replay, membership upsert, manual retry) write to an `audit_log` table with actor, org, action, target, and JSONB details. Writes are best-effort: audit failures log warnings but never block the user's operation.

5 Reliability & Concurrency

5.1 Delivery Guarantee

The system provides **at-least-once delivery** with idempotent handlers. True exactly-once is impossible across DB + broker + external service boundaries. The defence-in-depth stack:

5.2 Concurrency Proof Tests

- **Single-winner:** 8 workers race one job via `join_all`; exactly 1 succeeds.

Failure	Detection	Recovery
API commits but publish fails	Outbox row persists	Relay retries with backoff
Relay publishes then crashes	Lease expires	Another relay reclaims; JetStream dedup suppresses duplicate
Worker crashes mid-job	Lease expiry + JetStream re-delivery	Another worker claims with epoch+1; stale worker fenced
Worker cannot renew lease 3×	Consecutive renewal errors	Worker self-fences; refuses to commit blind
Handler panics or hangs	<code>catch_unwind</code> + timeout	Retryable failure; consumer task survives
Queue paused	Claim returns <code>QueuePaused</code>	NAK with delay; message redelivered when resumed
UNKNOWN external outcome	Reconciler after grace period	Policy: DLQ (default), redrive, or complete

- **Capacity:** 12 workers race on a capacity-3 queue; exactly 3 claim, the rest receive `QueueAtCapacity`.
- **Race harness:** 3 replicas × 40 jobs through a shared durable; zero double-execution (one ledger row per job), distribution across replicas, no leaked capacity tokens.

5.3 Handler Safety

Handlers execute inside `catch_unwind` and a hard timeout (`HANDLER_TIMEOUT_SECS`). A panic converts to retryable `handler_panicked`; a hang becomes `handler_timeout`. Both are safe under the idempotency contract. Handlers dispatch onto a semaphore-bounded pool (`WORKER_CONCURRENCY`) one slow job cannot starve its queue.

6 Frontend & UX

The dashboard is a React SPA (Vite) with a visionOS-inspired dark glassmorphism design: translucent blurred panels, neon accent colours, and layered spatial depth (live metrics on the closest layer, history behind).

- **Live updates:** project-scoped SSE stream drives instant refreshes when any job count changes; slow-poll fallback when SSE is unavailable.
- **Charts:** throughput time-series (completed vs failed per minute), status donut, capacity utilisation ring, lifecycle pipeline strip all clickable to filter the job list.
- **DLQ management:** per-entry Replay and AI-generated failure analysis viewer.
- **Accessibility:** axe-enforced zero critical/serious violations including color contrast; ARIA labels, keyboard navigation, screen-reader support.
- **Testing:** 7 vitest unit tests (token rotation, formatters, crash containment) + 14 Playwright browser e2e tests against the real API + 4 visual-regression baselines.

7 API Design

The stable contract is mounted at `/api/v1/*` with frozen unversioned aliases. 37 endpoints are documented in `docs/openapi.json` (41 schemas), programmatically cross-checked against route registrations.

7.1 Conventions

- **Error envelope:** `{error:{code, message}, request_id}` with machine-readable codes (NotFound, Validation, RateLimited, PayloadTooLarge,)

- **Pagination:** `page + page_size` (clamped 1100); responses carry `total` and `total_pages`
- **Idempotency:** `Idempotency-Key` header beats body `idempotency_key`; duplicates return 409 Conflict
- **Verb policy:** PATCH mutates stored state; POST verb-subresources are actions only; superseded endpoints emit RFC-8594 Deprecation/Sunset headers
- **Timezone:** all timestamps are UTC (RFC 3339); the frontend renders in the user's timezone

8 Documentation

- **README.md:** project overview, quickstart, reliability model, live demo link
- **docs/architecture.md:** component responsibilities, message flow, failure-recovery table, NATS subjects, deployment model
- **docs/design-decisions.md:** 15+ ADRs covering PostgreSQL vs Redis, NATS vs Kafka, outbox pattern, lease fencing vs Redlock, advisory lock vs etcd, token model, rate limiting, audit trail, hot/cold separation, handler safety, testing isolation, API versioning
- **docs/er-diagram.md:** Mermaid ER diagram with 28 entities, key relationships, index inventory, lifecycle narrative
- **docs/api.md:** REST route reference with parameter descriptions
- **docs/openapi.json:** OpenAPI 3.0 spec (37 paths, 41 schemas) programmatically cross-checked against route registrations
- **docs/testing.md:** test layer map, runnable commands, isolation design, CI topology
- **.env.example:** all 20+ configuration knobs documented
- **SECURITY.md:** authentication model, input handling, transport security, scope boundaries, vulnerability reporting

9 Testing

Layer	Location	Count	Runtime
Domain unit tests	<code>domain/src</code>	28	<1 s
Shared unit tests	<code>common/src</code> , <code>api/src</code> , <code>worker/src</code>	26	<2 s
DB integration	<code>db/src/integration_tests.rs</code>	12	~5 s
Pipeline e2e	<code>worker/tests/pipeline_e2e.rs</code>	1	~5 s
Race harness	<code>worker/tests/race_harness.rs</code>	1	~32 s
Archive bench	<code>db/tests/archive_bench.rs</code>	1	~2 s
Frontend unit	<code>frontend/src</code>	7	~4 s
Browser e2e	<code>frontend/tests/e2e</code>	14	~40 s
Visual regression	<code>frontend/tests/e2e</code>	4	~15 s
Adversarial payloads	<code>scripts/payload_attacks.py</code>	51	~30 s

9.1 Key Design Decisions

Per-test databases

Each integration test creates a throwaway `js_test_*` database and applies all migrations twice, proving idempotency continuously. No truncate races; full parallelism.

No mocks below unit layer

DB integration tests and pipeline e2e speak to real PostgreSQL and real JetStream. The only stubs are fetch-level doubles in the frontend refresh-wrapper test.

CI topology

Four gating jobs: `test` (build + test + clippy + vitest + npm audit), `pipeline-e2e` (pipeline

+ race harness + archive bench), **frontend-e2e** (Playwright against real API), **coverage** (llvm-cov summary). None are allow-failure.

10 Deployment

The system deploys as a single Docker container on Render.com's free tier, running NATS, API, worker, and scheduler behind a supervisor. PostgreSQL is provisioned separately as a managed database. The live demo is available at <https://jobflow-nlwj.onrender.com>.

```
# render.yaml blueprint
services:
  - type: web
    name: jobflow
    runtime: docker
    dockerfilePath: ./deploy/Dockerfile.all-in-one
    plan: free
    healthCheckPath: /health
databases:
  - name: jobflow-db
    plan: free
```

11 Conclusion

Jobflow demonstrates production-grade engineering across every layer of a distributed system: a provably correct state machine, atomic concurrency primitives with test-backed invariants, defence-in-depth error handling, comprehensive observability, and a polished user interface. Every design decision is documented with its trade-offs, every critical path is covered by automated tests that run in CI, and the live deployment validates the full pipeline from HTTP request to job execution to failure recovery.